# claude-windows / 8d4866bf-8edd-4da9-909e-022ddee12675

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\8d4866bf-8edd-4da9-909e-022ddee12675\subagents\workflows\wf_684a0618-ccb\agent-a63bed7936001a4c5.jsonl`
- SHA-256: `8275c4d9e2d36b079fa8b1c03c674d48676bd73dbc4ab8ee96b594fa975184ec`
- Source modified: `2026-06-19T19:18:30+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_684a0618-ccb`
- session_id: `8d4866bf-8edd-4da9-909e-022ddee12675`

## Transcript

#### 1. user (2026-06-19T19:17:29.043Z)

THE IDEA TO ASSESS: a desktop app OR web UI where the user points KhelSutra at an ONLINE storage location they already own/pay for ? a Google Cloud Storage bucket, Google Drive, OneDrive/Microsoft 365, Dropbox, S3, etc. KhelSutra then WATCHES that folder, pulls/streams new recordings as they arrive, and AUTO-prepares highlights (same processing as the local/desktop path). "Bring your own storage + autosync": footage lands in their cloud ? reels appear automatically, zero manual upload.

You are assessing how this idea fits ONE area of the KhelSutra (badminton-highlight-indexer) docs. Read: docs/STORAGE_SHARING_MODEL.md and docs/GOLDEN_DATA_SHARING.md. GOLDEN_DATA_SHARING already uses a OneDrive shared folder that auto-syncs both ways (GPU box ? CPU sessions) ? surface that as an existing precedent for cloud-folder autosync. What does the storage/sharing model say about where user video lives and read-only access?

Be evidence-based and honest ? quote real anchors, don't invent. If the doc does NOT anticipate the idea, say so plainly and note what it WOULD take. Return the structured object.

#### 2. assistant (2026-06-19T19:17:31.598Z)

I'll read both docs to assess how the BYO-storage/autosync idea fits.

#### 3. user (2026-06-19T19:17:32.612Z)

1 # Storage-priced sharing ? mental model (DRAFT v2 for co-design)
2
3 **Status:** DRAFT ? owner + Claude co-design session pending; nothing here is ratified.
4 **Date:** 2026-06-12 (v2 adds: model-tier axis, social layer, on-device end-state, abuse
5 economics ? per owner's second brief) · **Source:** owner briefs (voice note `Umiya Velociti.m4a`
6 pending transcription) + UI_PROPOSAL · HOSTING_PLAN · PMF research · NEXT_STEPS quality table.
7 ? If ratified, this **supersedes the monetization framing in COMMERCIALIZATION.md** (away from
8 per-video/subscription, toward storage-anchored tiers that bundle model quality) ? record as an ADR.
9
10 ## 1. The one-line model (v2)
11
12 Don't sell videos or subscriptions ? **sell shelf space, and let model quality ride the shelf**:
13 free = **2 GB + the owned model**; paid = **more shelf + the best model**; the visible meter is
14 always storage. The social layer makes the shelf worth filling; the owned model makes the free
15 tier cost ~nothing; the end-state moves compute onto the customer's own hardware

entirely.

## 1a. The three coupled axes (one user-visible meter)

| Axis | Free | Paid | End-state (serious users) |
|---|---|---|---|
| **Storage** (the meter) | 2 GB library | 10 GB+ / pooled | Mostly local disk; cloud = shareable library only |
| **Model** (rides the tier) | Owned model only ? **never Gemini** | Best path (two-stage refine / frontier) | Best owned model, on-device |
| **Compute** (the cost) | Our GPU, capped runs | Our GPU, priority | **Their GPU** (USB-drive workflow) |

## 2. The load-bearing decision: two tiers of object

| Tier | What | Size reality | Stored? | Quota? |
|---|---|---|---|---|
| **Source** (raw match upload) | Processing INPUT | 1?4 GB each | Temporarily: kept through processing + correction window, then **auto-deleted** (e.g. 7 days or on "corrections confirmed") | Not by default; optional "keep my raw match" toggle = counts |
| **Library** (compiled reels, kept rally clips, thumbnails) | The PRODUCT | reel ? 50?80 MB (3?5 min 720p) | Durable | **Yes ? this is what 2 GB means** |

Why this split: 2 GB cannot hold even one raw match, but holds **~25?40 reels** ? the quota is only meaningful against outputs. It also kills the "free CDN" abuse vector for a storage-priced product: **the library only ever contains pipeline outputs** ? there is no path to park arbitrary files in it. (Sources additionally pass the existing relevance validator.)

So: *"Do we store the uploaded video?"* ? **Yes, transiently ? it's a processing input with a retention deadline, not a possession.** The reel is the possession.

## 3. Eviction policies = a pluggable registry (the repo's own pattern)

`EvictionPolicy` ABC + registry, exactly like `segmenter_registry`:
`fifo` (oldest first) · `largest_first` · `smallest_first` · `least_visited` (view-counter) ·
`hard_stop` (no deletion ? block new saves until user frees space or upgrades).

- **Proposed default: `hard_stop`** ? never silently delete someone's match memories; FIFO etc. are opt-in conveniences. (Co-design question #2.)
- **Grace flow:** over quota ? banner + email with the exact list the chosen policy would evict ? N-day grace (propose 7) ? evict, logged in an `eviction_log`. Offer download-before-evict.
- `least_visited` needs a decision: owner views, public views, or both?

## 4. CUJ after upload (end-to-end)

```
record ? upload source ? async job (PR-1) ? review/correct (corpus engine, PR-4)
   ? compile reel(s) ? reel lands in LIBRARY (quota meter ticks)
   ? share: profile page / unlisted link / private
   ? source hits retention deadline ? auto-delete
   ? library overflow ? eviction policy (with grace)
```

A Gmail-style **quota meter** is permanent UI. Profile = `khelsutra.guru/@handle`: reel gallery, view counts (feeds `least_visited`), sport tags. Per-reel visibility: public / unlisted / private.

## 5. Upload path: "500 MB cap" vs "point at a bucket"

- **Alpha (tunnel ? local box):** keep the PR-2 chunked endpoint. A 500 MB cap ? 10?15 min of 1080p ? fine for pre-trimmed games, **kills full matches (1?4 GB)**. Recommend keeping the 4 GB chunked cap and rate-limiting per user instead; tomorrow's CUJ run will tell us real file sizes. (Co-design question #3.)
- **Beta (hosted):** the bucket IS the upload path ? browser ? object storage **directly** via signed resumable URLs (PLATFORM P2 `StorageBackend`); the server never proxies bytes; "max filesize" becomes policy enforced at URL-issue + post-upload validation, not a proxy limit.

## 6. The real bill is egress, not storage (sharing = bandwidth)

Storing a free user's 2 GB costs **< ?5/month anywhere**. Serving is the risk: on GCS, egress is ~$0.12/GB ? one mildly viral reel (80 MB × 1,000 views) ? **$10 of bandwidth**; on Cloudflare R2, egress is **$0**. ? Proposed hybrid: **sources ? GCS** (credits, lifecycle auto-delete does the retention for free) · **library/serving ? R2** (zero egress; virality becomes free marketing instead of a bill). Storage-anchored pricing only has margins on zero-egress serving.

## 7. Pricing sketch (INR, storage-anchored ? numbers are placeholders)

Free **2 GB** ? paid tiers, e.g. +10 GB / +50 GB monthly or yearly. Anchors: Google One is ?130/mo for 100 GB (we are not selling bytes ? we sell *organized, shareable match memory* ? priced above raw storage but inside the PMF envelope: B2C $60?180/yr). Academy (B2B overlay): **pooled storage** across students + coach-managed profiles. Processing cost note: Gemini path is ~$0.05/run ? decide whether the free tier includes unlimited processing or N matches/month (quota is storage-anchored, but compute isn't free ? soft cap recommended).

## 7a. The model-tier axis ? why "free never touches Gemini" is load-bearing

1. **Free-tier economics:** Gemini ? $0.05/run forever; the owned model is ~$0 marginal on owned GPU. A free tier on the owned model is *sustainable at scale*; one on Gemini bleeds.
2. **The corpus flywheel becomes the business model:** free users correct rallies (PR-4) ? golden corpus grows ? owned model climbs the quality table (heuristic 0.48 ? Gen-0 0.74 ? target ? wrapper's 0.66 on multi-court) ? free tier literally improves every quarter ? more users ? more corrections. The owned-model roadmap (OWNED_MODEL_IMPLEMENTATION_PLAN) IS the free-tier product engine.
3. **Independence & privacy:** free tier immune to provider 503 waves, price hikes, and terms drift; paid tier uses Gemini only on its paid/private tier per existing guardrails.
4. **Honest quality ladder (from NEXT_STEPS):** free = owned (today Gen-0-grade, improving); mid = two-stage WASB+refine (0.83 clean); top = best available path (wrapper 0.93 clean). Paid stays ahead until the owned model overtakes ? at which point margins improve

again.

## 7b. Tier sketch (names placeholder ? co-design)

| | **Rally** (free) | **Smash** (paid) | **Court** (academy/pro) |
|---|---|---|---|
| Library | 2 GB | 10 GB+ | Pooled across students |
| Model | Owned only | Best path + priority queue | Best owned, **on-device** |
| Processing | N matches/mo cap | Higher cap | Unlimited (their hardware) |
| Social | Full | Full | + coach overlay, athlete profiles |

## 7c. Social layer ? "Facebook of (match) videos," scoped honestly

The shelf is worth paying for only if filling it is social. **Interactions are free; storage is the meter** ? never paywall the social loop itself.
- **v1 social (already in the alpha/beta plan):** profile `@handle`, reel gallery, view counts, share links. **v2 social (post-PMF, big build):** follows, reactions (badminton-native: ? "clean smash"), comments, tag-a-player (consent-gated), academy groups, "rally of the week."
- Network effect is local-first: the same court's regulars find each other ? academy = built-in graph seed.
- ? Scope discipline: feed/moderation/notifications is a product in itself; it lands AFTER the alpha proves retention, not alongside it.

## 7d. On-device end-state ? the USB-drive vision

Serious users plug a drive of match videos into **their** Windows/GPU machine; the app processes locally (this repo already *is* that pipeline) and syncs only reels + metadata to the cloud library/profile. Cloud cost for the heaviest users ? storage + social only.
- **The licensing guardrail becomes the product moat:** Gemini can never ship on-device; the owned model (Qwen3-VL base, Apache-2.0 / MIT-only deps) CAN. Commercial-clean licensing was the prerequisite all along.
- Staging: academy pilot boxes first (serious, GPU-equipped, exactly the owner's own setup) ? consumer Windows app later (needs the WSL-free Linux-native/DirectML detector path ? the detector ABC seam exists for this).
- Maps to PLATFORM P3 `local`/`byo_worker` compute backends ? productized.

## 7e. Abuse economics (why "serious pay, abusers can't extract value")

Each axis is self-defending: **storage** ? library accepts only pipeline outputs (no arbitrary files), sources auto-expire; **compute** ? free tier = owned model (cheap) + N runs/mo cap + relevance validator rejects non-sport uploads; **bandwidth** ? R2 zero-egress serving + signed, revocable links (and the egress allowance baked into paid storage pricing per §6: price/GB = storage + ?·expected-views·reel-size·egress-rate + margin; on R2 the ?-term ? 0 and becomes margin); **social** ? report/takedown + rate limits. There is no tier where an abuser extracts more value than they pay for, and no free capability that costs us marginal money at scale.

## 8. Guardrails (carry-overs, non-negotiable)

Consent-at-capture for other players (Phase-2 recording rules) · **minors never on public
profiles** and never in the training pool · report/takedown path on public reels · share links
signed + revocable · training-data opt-in stays separate from storage/sharing consent.

## 9. Schema deltas (when we build ? not before ratification)

`users(quota_bytes, eviction_policy)` · `library_objects(id, owner, video_id, kind?{reel,clip,thumb},
bytes, created_at, last_visited_at, visit_count, visibility, storage_ref)` · `eviction_log` ·
source uploads gain `retention_deadline`. `storage_ref` follows P2's `StorageRef` (backend id + key).

## 10. Co-design agenda (the decisions, in order)

**Object model & policies**
1. Ratify the **two-tier model** (sources transient / library durable)?
2. **Default eviction:** `hard_stop` (proposed) vs `fifo`? Grace days = 7?
3. Quota counts reels only, or also opt-in kept sources? (proposed: both, sources opt-in)
4. `least_visited`: whose visits count?

**Tiers & pricing**
5. Ratify **"free never touches Gemini"** (owned-model-only free tier)?
6. Free tier processing cap: N matches/month = ?
7. Tier names/sizes (Rally 2 GB / Smash 10 GB / Court pooled ? placeholders) + INR price points.
8. Egress-baked pricing accepted (§7e formula), with R2-for-library hybrid despite GCP credits?

**Upload & serving**
9. **Alpha upload cap:** 500 MB vs keep 4 GB chunked? (use the first real CUJ file sizes)

**Social & end-state**
10. v1 social scope = profiles + share links + view counts ONLY (feed/comments post-PMF)?
11. Minors on public profiles: propose **hard exclusion** in v1 (stricter than the training-pool
    rule ? public sharing is a different risk class). Ratify?
12. Profile namespace: `@handles` on khelsutra.guru?
13. On-device milestone placement: academy pilot boxes first, consumer app after the WSL-free
    detector path?

**Paper trail**
14. ADR to supersede COMMERCIALIZATION.md's subscription framing once 1?13 settle.

### 4. user (2026-06-19T19:17:32.705Z)

# Golden / GPU-Extractor Data Sharing ? GPU box ? dev sessions

**Status:** convention + plan (2026-06-15). Tracked in **#175**.
**Purpose:** a single cloud-synced location for the GPU-extracted golden artifacts (trajectory CSVs, `*_golden_features.json`, manifests, baseline snapshots) so the **GPU box** and **CPU dev/agent sessions** share the same files ? without committing large data to git.

## Why this exists
The rally-quality measurement is split across two machines by capability:
- **`backend/eval/fusion_golden.py`** (shuttle+person feature **extraction**) needs the

**videos**, the **trajectory CSVs**, the **`.rallies.csv` golden labels**, and a **GPU/WSL** stack ? only the GPU box can run it.
- **`backend/eval/fusion_compare.py`** / **`backend/eval/ablation.py`** (the LOVO **litmus**: ?F1, bootstrap CI, paired sign test) are **pure-CPU re-scoring** that read only the small `*_golden_features.json` ? any session (incl. a CPU dev container) can run them.

Today those artifacts sit in gitignored `output/` (per-machine) and scattered local folders (`...\Annotation Setup\Collect\Trajectories\`, `...\Golden Labelled\`). Nothing syncs them between machines, so a CPU session cannot see the GPU box's extraction. This wires that gap with OneDrive.

## Canonical shared folder
`%USERPROFILE%\OneDrive\rally-annotator\golden-data\`  ? `C:\Users\avidu\OneDrive\rally-annotator\golden-data\`

OneDrive is present + signed-in on both the GPU box and the dev machine under the same account, so it auto-syncs. Layout (date/size-tagged so a re-extraction is a new folder, never an overwrite):

```
golden-data/
  README.md
  features/<corpus-tag>/       *_golden_features.json     (small ? the key shared artifact)
  trajectories/<corpus-tag>/   *_predict.csv / *_wasb.csv (medium ? reusable across runs)
  manifests/                   *_lovo_manifest.json
  baselines/                   leaderboard / regression snapshots (or keep tracked in eval_baselines/)
```

`<corpus-tag>` = e.g. `2026-06-15-n6` (date + corpus size). **Do NOT** put raw videos here (tens of GB) ? keep those local to the GPU box.

## The GPU ? CPU workflow
1. **GPU box** (videos + CSVs + labels present):
   ```
   python -m backend.eval.fusion_golden --manifest <manifest> \
     --out-dir "%USERPROFILE%\OneDrive\rally-annotator\golden-data\features\2026-06-15-n6"
   ```
   (or extract to local `output/` then copy the `*_golden_features.json` into the shared folder).
2. OneDrive syncs the JSONs to the dev machine automatically (small files, seconds).
3. **CPU dev/agent session** points the analysis at the shared folder:
   ```
   python -m backend.eval.fusion_compare --features-dir "...\golden-data\features\2026-06-15-n6" --record
   python -m backend.eval.ablation       --features-dir "...\golden-data\features\2026-06-15-n6" --granularity group
   ```
   ? real person-feature ?F1 / CI / sign test. Copy the JSONs into the repo's `output/` to also light up the `skipif` golden-litmus tests (`tests/test_ablation.py::test_litmus_reproduces_gate_a`, `tests/test_fusion_compare.py`).

## Optional code ergonomics (in #175, not required)
- A `RALLY_GOLDEN_DIR` env var (default `output/`) honored by `fusion_golden` / `fusion_compare` / `ablation` so neither machine hardcodes the path. (The CLI `--out-dir` / `--features-dir` flags already make this a convention; the env var is just ergonomics.)
- A tiny `scripts/sync_golden.py` to copy `output/*_golden_features.json` ? the shared folder.

## Guardrails
- The shared folder lives **outside the repo** ? never committed. Keep `output/` gitignored and do **not** symlink the shared folder into a tracked path.

50 - The features derive from the **Proprietary** corpus ? keep the OneDrive folder **private** (no public share links).
51
52 See also: `docs/NEXT_STEPS.md` (the "Next-session pickup" anchor), `docs/MULTI_SIGNAL_FUSION_PLAN.md` (P3), `docs/TRACKNET_WSL_SETUP.md` (the GPU/WSL detector setup), and **`docs/GOLDEN_REGRESSION_FIXTURES.md`** ? the COMMITTED, de-attributed regression **subset** (synthetic + owner-cleared) that runs in-repo *without* the videos. This OneDrive folder stays the **full private corpus**; the committed subset is the minimized/anonymized, shuttle-only public tier (no person/audio, opaque surrogate ids).
53

### 5. assistant (2026-06-19T19:17:35.599Z)

I have both docs read in full. This is a clean, evidence-based assessment ? both files are short enough to quote precisely. Let me compose the structured answer.

### 6. user (2026-06-19T19:18:17.572Z)

Structured output provided successfully

### 7. assistant (2026-06-19T19:18:30.869Z)

I've completed the assessment. Summary of what I found across the two docs:

**Existing autosync precedent (confirmed):** `docs/GOLDEN_DATA_SHARING.md` already runs a real cloud-folder autosync ? `%USERPROFILE%\OneDrive\rally-annotator\golden-data\` is "present + signed-in on both the GPU box and the dev machine under the same account, so it auto-syncs," moving GPU-extracted artifacts to CPU sessions "automatically (small files, seconds)," with a `scripts/sync_golden.py` copy helper. So watch-a-cloud-folder is a lived pattern in the repo ? but it's an *internal, same-account, small-JSON* sync, and its guardrail ("keep the OneDrive folder private ? Proprietary corpus, no public share links") is itself a signal that even internal cloud-folder sync is handled as privacy-sensitive.

**Where user video lives / read-only access (the core question):** `docs/STORAGE_SHARING_MODEL.md` §2 is explicit ? raw match video is a **Source** tier object: a transient processing INPUT (1?4 GB), "kept through processing + correction window, then auto-deleted," "not a possession." The **Library** (reels) is the durable product and the only thing the 2 GB meter counts. Critically, the doc assumes sources live in **KhelSutra-controlled** storage. The only "point at a bucket" language (§5 Beta) is *our* object storage as the upload TARGET via signed resumable URLs ? a browser-push path, **not** a pull/watch of a folder the customer already owns. There is no read-only "connected external source" concept anywhere.

**Verdict:** partial anticipation. The idea is the natural autosync front-door to the §7d "BYO storage / their GPU" end-state and slots in as a new **read-only StorageBackend (PLATFORM P2) + watcher mode** feeding the existing §2 source tier and §4 CUJ. What's missing: the watcher/change-feed itself, an OAuth/least-privilege scope model, a "read-only external source" object in the §2/§9 schema, and ? most load-bearing ? an egress/compute-placement decision (pulling 1?4 GB out of the user's cloud into our GPU is the inverse of the §6 serving-egress concern the whole pricing model is built around).

Full evidence, anchors, gaps, and risks are in the structured object.